

4.<Źródło>Algorytmy optymalizacji - programowanie dynamiczne

Jasne, oto profesjonalne opracowanie na temat programowania dynamicznego, przygotowane na podstawie dostarczonych materiałów, z uwzględnieniem formalnego tonu i notacji matematycznej.

Algorytmy Optymalizacji: Programowanie Dynamiczne

Wprowadzenie

Programowanie dynamiczne (PD) jest zaawansowaną techniką projektowania algorytmów, która służy do rozwiązywania złożonych problemów optymalizacyjnych. Kluczowe podejście polega na dekompozycji problemu głównego na serię mniejszych, nakładających się na siebie podproblemów. Rozwiązania tych mniejszych podproblemów są zapamiętywane (memoizacja) i wykorzystywane do konstrukcji rozwiązania problemu nadrzędnego.

Metoda ta jest szczególnie skuteczna w przypadku problemów, które wykazują dwie fundamentalne właściwości:

1. **Nakładające się podproblemy (Overlapping Subproblems):** Algorytm wielokrotnie napotyka i rozwiązuje te same podproblemy. PD rozwiązuje każdy z nich tylko raz.
2. **Optymalna podstruktura (Optimal Substructure):** Optymalne rozwiązanie całego problemu można skonstruować z optymalnych rozwiązań jego podproblemów.

Podstawy Schematu i Zasada Bellmana

Schemat programowania dynamicznego przekształca zadanie optymalizacyjne w wieloetapowy proces decyzyjny.

- **Stan** procesu na etapie k oznaczamy jako S_k .
- **Decyzja** podejmowana na etapie k to d_k .

- **Transformacja stanu** opisuje przejście między stanami:

$$S_k = T(S_{k-1}, d_{k-1}).$$

Celem jest znalezienie takiej sekwencji decyzji (strategii) $d_1, d_2, \dots, d_N$, która optymalizuje (minimalizuje lub maksymalizuje) globalną **funkcję celu** $F(S_1, \dots, S_N; d_1, \dots, d_N)$.

Kluczowym elementem, który umożliwia zastosowanie PD, jest **własność Markowa**, która stanowi, że stan na etapie k zależy wyłącznie od stanu na etapie $k - 1$ i decyzji d_{k-1} , a nie od całej historii procesu.

Własność ta prowadzi bezpośrednio do **Zasady Optymalności Bellmana**:

Niezależnie od stanu początkowego i początkowej decyzji, pozostałe decyzje muszą stanowić optymalną strategię w odniesieniu do stanu wynikającego z pierwszej decyzji.

Dla addytywnej funkcji celu, postaci $F = \sum_{i=1}^N f_i(S_i, d_i)$, zasadę tę można sformalizować. Definiując $q_k(S_k)$ jako optymalną wartość funkcji celu od etapu k do końca, przy stanie początkowym S_k , otrzymujemy fundamentalne **równanie rekurencyjne Bellmana**:

$$q_k(S_k) = \min_{d_k} (f_k(S_k, d_k) + q_{k+1}(T(S_k, d_k)))$$

z warunkiem brzegowym $q_{N+1}(\cdot) = 0$. Równanie to rozwiązuje się parametrycznie (zazwyczaj wstecz), od etapu N do 1.

Przykłady Zastosowań

Poniżej przedstawiono klasyczne problemy optymalizacyjne rozwiązywane za pomocą programowania dynamicznego.

1. Najdłuższa Droga w Grafie Acyklicznym (DAG)

Problem ten, znany w zarządzaniu projektami jako **Metoda Ścieżki Krytycznej (CPM)**, polega na znalezieniu w skierowanym grafie acyklicznym $G = (V, E)$ drogi o maksymalnej sumie wag wierzchołków.

- **Definicja podproblemu:** Niech d_i oznacza długość najdłuższej ścieżki kończącej się w wierzchołku $i \in V$.
- **Wagi:** Każdy wierzchołek i (reprezentujący zadanie/czynność) ma wagę $p_i > 0$ (czas trwania).
- **Równanie rekurencyjne:** Długość najdłuższej ścieżki do wierzchołka i jest sumą jego wagi oraz maksimum z długości najdłuższych ścieżek dochodzących do jego bezpośrednich poprzedników.

$$d_i = p_i + \max_{j \in B_i} \{d_j\}$$

gdzie $B_i = \{j \in V \mid (j, i) \in E\}$ to zbiór bezpośrednich poprzedników wierzchołka i .

- **Warunek początkowy:** Dla wierzchołków bez poprzedników ($B_i = \emptyset$), $d_i = p_i$.
- **Złożoność:** Algorytm ma złożoność wielomianową $O(|V| + |E|)$, ponieważ wymaga jednokrotnego przetworzenia każdego wierzchołka i krawędzi w porządku topologicznym.

2. Problem Plecakowy (Knapsack Problem)

Problem polega na wyborze podzbioru przedmiotów o maksymalnej łącznej wartości, których łączna waga nie przekracza pojemności plecaka.

- **Dane:** Zbiór N przedmiotów, gdzie przedmiot i ma wartość c_i i wagę a_i . Pojemność plecaka wynosi b .
- **Cel:** Maksymalizować $\sum_{i \in I} c_i$ przy ograniczeniu $\sum_{i \in I} a_i \leq b$, gdzie $I \subseteq N$.
- **Definicja podproblemu:** $F(i, y)$ to maksymalna wartość, jaką można uzyskać, wybierając spośród pierwszych i przedmiotów, przy pojemności plecaka równej y .
- **Równanie rekurencyjne:** Dla każdego przedmiotu i mamy dwie możliwości: nie bierzemy go lub bierzemy go (jeśli się mieści).

$$F(i, y) = \begin{cases} F(i-1, y) & \text{jeśli } a_i > y \\ \max\{F(i-1, y), c_i + F(i-1, y - a_i)\} & \text{jeśli } a_i \leq y \end{cases}$$

- **Złożoność:** Złożoność obliczeniowa wynosi $O(nb)$, co klasyfikuje algorytm jako pseudowielomianowy (złożoność zależy od wartości liczbowej danych wejściowych, a nie tylko od ich rozmiaru).

3. Problem Komiwożera (Traveling Salesperson Problem - TSP)

TSP jest problemem NP-trudnym, polegającym na znalezieniu najkrótszej trasy (cyklu Hamiltona) odwiedzającej zbiór miast. Algorytm Helda-Karpa jest jego rozwiązaniem opartym na PD.

- **Definicja podproblemu:** $D(S, p)$ to długość najkrótszej ścieżki startującej z miasta 1, odwiedzającej wszystkie miasta w zbiorze $S \subseteq N$, i kończącej się w mieście $p \in S$.
- **Równanie rekurencyjne:** Aby dotrzeć do miasta p jako ostatniego na trasie przez zbiór S , musieliśmy przybyć z jakiegoś innego miasta $k \in S \setminus \{p\}$.

$$D(S, p) = \min_{k \in S \setminus \{p\}} \{D(S \setminus \{p\}, k) + d_{kp}\}$$

gdzie d_{kp} to odległość między miastami k i p .

- **Warunek początkowy:** $D(\{1\}, 1) = 0$.
- **Złożoność:** Złożoność czasowa i pamięciowa jest wykładnicza, rzędu $O(n^2 2^n)$, co czyni algorytm praktycznym tylko dla niewielkiej liczby miast.

4. Problem Szeregowania na Jednej Maszynie

Jest to złożony problem, gdzie celem jest znalezienie sekwencji wykonywania zadań, aby zminimalizować sumę kosztów zależnych od terminów ich ukończenia.

- **Definicja podproblemu (wersja uproszczona):** $F(I)$ to minimalny koszt uszeregowania zadań ze zbioru $I \subseteq J$.
- **Równanie rekurencyjne:** Zakładając, że któreś z zadań $k \in I$ musi być wykonane jako ostatnie, jego czas ukończenia wyniesie $p(I) = \sum_{j \in I} p_j$.

$$F(I) = \min_{k \in I} [F(I \setminus \{k\}) + f_k(p(I))]$$

gdzie $f_k(\cdot)$ to funkcja kosztu dla zadania k .

- **Implementacja i złożoność:** Stany (podzbiory I) można efektywnie reprezentować za pomocą masek bitowych. Złożoność jest wykładnicza, $O(n 2^n)$, ze względu na konieczność obliczenia wartości $F(I)$ dla wszystkich 2^n podzbiorów. Wprowadzenie ograniczeń (np. relacji

poprzedzania, terminów gotowości) dodatkowo komplikuje model i zwiększa złożoność.

5. Inne Algorytmy Powiązane z PD

Prezentacja wymienia również inne fundamentalne algorytmy, które opierają się na podobnych zasadach rekurencji i optymalnej podstruktury.

- **Algorytm Bellmana-Forda:** Klasyczny algorytm PD do znajdowania najkrótszych ścieżek z jednego źródła w grafach z wagami ujemnymi. Jego złożoność to $O(|V| \cdot |E|)$.
- **Algorytm Floyd-Warshalla:** Algorytm PD do znajdowania najkrótszych ścieżek między wszystkimi parami wierzchołków. Złożoność wynosi $O(|V|^3)$.
- **Problem Wydawania Reszty:** Celem jest wydanie danej kwoty K przy użyciu minimalnej liczby monet. Niech $f_j(L)$ będzie minimalną liczbą monet potrzebną do wydania kwoty L przy użyciu pierwszych j nominałów.

$$f_j(L) = \min\{f_{j-1}(L), 1 + f_j(L - w_j)\}$$

Złożoność wynosi $O(nK)$, gdzie n to liczba nominałów.

Warto zauważyć, że **Algorytm Dijkstry**, choć oparty na relaksacji krawędzi podobnej do PD, jest klasyfikowany jako **algorytm zachłanny**, ponieważ na każdym kroku wybiera lokalnie optymalną decyzję (przetwarza najbliższy, jeszcze nieodwiedzony wierzchołek).

Podsumowanie

Programowanie dynamiczne jest potężnym paradygmatem rozwiązywania problemów optymalizacyjnych, które posiadają własność optymalnej podstruktury i nakładających się podproblemów. Jego siła leży w systematycznym budowaniu rozwiązania globalnego na podstawie wcześniej obliczonych i zapamiętanych rozwiązań mniejszych problemów. Głównym ograniczeniem jest często wykładnicza złożoność czasowa i pamięciowa, znana jako "przekleństwo wymiarowości" (curse of dimensionality), która wynika z dużej liczby możliwych stanów w procesie decyzyjnym. Mimo to, dla

wielu kluczowych problemów, w tym tych o złożoności pseudowielomianowej, pozostaje ono najskuteczniejszą znaną metodą.